Decision Tree

Saturday, March 14, 2026 12:58 PM

Decision Tree:(Recursive Binary splitting-greedy approach)-recursively it will do the every possibility

-Can perform both classification and regression model

-Root-child-binary-node will not further split (pure node)

-Impure node- can split again
-in a node it will only think about at that particular point of time(yes/node)(yes-pure)(no-impure) – greedy search
-if model extends largley it will become overfitting

To reduce that we use tree pruning

Post-pruning-backward pruning -non sig /non performing branches are removed – expensive on computation
Pre-pruning – Forward pruning – while building checks significance-stops non sig branches from generating

Pre-pruning- Gini index-
Entropy – mesure of randomness in the data
Information gain – ento PRoot not – wieght of child nodes – based on that we get higher split

Decision Tree (Recursive Binary Splitting – Greedy Approach)

- Decision Tree uses **recursive binary splitting** to divide the dataset.
- At each step it checks **all possible splits** and selects the best one.
- It follows a **greedy approach**
- It only looks for the **best split at the current step**
- It does **not consider future splits**
- Can be used for **both**:
- **Classification** (predict categories like spam/not spam)
- **Regression** (predict continuous values like price)

Tree Structure

- **Root node** – starting point of the tree
- **Child nodes** – nodes created after splitting
- **Binary split** – each split usually creates **two branches**
- **Pure node**
- Contains only **one class of data**
- Cannot be split further
- **Impure node**
- Contains **mixed classes**
- Can be **split again**

Greedy Decision at Each Node

At every node the model asks:

- **Is this node pure?**
- Yes → stop splitting
- No → split further

The model only focuses on **the current node decision**, not the entire tree.

Overfitting Problem

If the tree **keeps growing too deep**, it will:

- memorize the training data
- become **overfitting**
- perform poorly on new data

Tree Pruning (Reducing Overfitting)

To control tree size we use **pruning**.

1. Pre-Pruning (Forward Pruning)

- Happens **during tree construction**
- The algorithm checks if a split is **significant**
- If not significant → **stop splitting**

This prevents unnecessary branches from being created.

2. Post-Pruning (Backward Pruning)

- Happens **after the tree is fully built**
- The algorithm removes **non-significant or weak branches**
- More **computationally expensive**

Information Gain means **how much useful information a split gives** in a decision tree.

- It tells us **how much uncertainty is reduced** after splitting the data.
- A decision tree chooses the split with the **highest information gain**.
- It is mainly based on **entropy**.

Formula:
Information Gain = Entropy(parent) - Weighted Entropy(children)

Simple meaning

- First check how mixed the parent data is.
- Then split the data.
- Check how mixed the child groups are.
- If the split makes the child groups much purer, then **information gain is high**.

Why we use it

- To find the **best feature** to split on.
- To reduce randomness in each step.
- To make the tree more accurate.

Tiny example

Suppose 10 students:

- 5 passed
- 5 failed

This is very mixed, so entropy is high.

Now split by **studied?**

Group 1:

- 4 passed, 1 failed

Group 2:

- 1 passed, 4 failed

Now the groups are much cleaner than before.

So this split gave a **high information gain**.

In one line

Information gain = reduction in disorder after splitting.

Gini or **Gini Impurity** measures **how mixed the classes are in a node**.

- It is used in a **decision tree** to check how good a split is.
- If the node has only one class, **Gini = 0**.
- If the classes are mixed, Gini value becomes **higher**.
- Lower Gini means the node is **more pure**.
- The decision tree chooses the split with **lower Gini impurity**.

Why we use it:

- To find the best split in the data
- To make child nodes more pure
- To improve prediction accuracy

Simple idea:

- **Low Gini** = good split
- **High Gini** = mixed data, bad split

Formula:

Gini=1−∑(pi)2Gini = 1 - \sum (p_i)^2Gini=1−∑(pi)2

where pip_ipi is the probability of each class.

Example:
If a node has only "Yes" class, Gini = **0**
If a node has 50% Yes and 50% No, Gini is **high**

In one line:
Gini tells how impure or mixed a node is. It is only used to split not lose the data it helps choosing the parent class and help building the decision tree

- **Entropy** measures the **disorder or impurity** in a node.
- It tells us how mixed the classes are.
- If all data in a node belongs to one class, entropy is **0**.
- If the classes are equally mixed, entropy is **high**.
- A decision tree tries to choose splits that **reduce entropy**.
- Lower entropy means a **purer node**.

Formula:

Entropy=−∑pilog2(pi)Entropy = -\sum p_i \log_2(p_i)Entropy=−∑pi log2 (pi)

where pip_ipi is the probability of each class.

Example

Suppose a node has 10 students:

- 5 Pass
- 5 Fail

So:

- p(Pass)=5/10=0.5p(\text{Pass}) = 5/10 = 0.5p(Pass)=5/10=0.5
- p(Fail)=5/10=0.5p(\text{Fail}) = 5/10 = 0.5p(Fail)=5/10=0.5
  Entropy=−(0.5log20.5+0.5log20.5)Entropy = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5)Entropy=−(0.5log2 0.5+0.5log2 0.5) Entropy=−(0.5(−1)+0.5(−1))Entropy = -(0.5(-1) + 0.5(-1))Entropy=−(0.5(−1)+0.5(−1)) Entropy=−(−1)=1Entropy = -(-1) = 1Entropy=−(−1)=1

So entropy = **1**, which means the node is highly mixed.

Pure node example

If all 10 are Pass:

- p(Pass)=1p(\text{Pass}) = 1p(Pass)=1
- p(Fail)=0p(\text{Fail}) = 0p(Fail)=0

Entropy = **0**

That means the node is perfectly pure.

Why we use it

- To measure impurity in a node
- To decide the best split in a decision tree
- To calculate **Information Gain**

One-line note:

- **Entropy = measure of randomness/disorder in a node.**

In classification

- We use DT when the target/output is a **category**

- Example: **Yes/No**, **Pass/Fail**, **Spam/Not Spam**
- The tree splits the data into groups based on conditions
- Final leaf node gives the **class label**
- It chooses splits using **Gini** or **Entropy**

Example:
Predict whether a customer will buy or not

- Age < 30
- Salary > 40k
- Previous purchase = Yes

Final output: **Buy = Yes**

Loss:

- In **classification**, loss is based on wrong class prediction, often **log loss**
- In **decision tree classification**, split quality is checked using **Gini** or **Entropy**

In regression

- We use DT when the target/output is a **continuous number**
- Example: **house price, salary, marks, sales**
- The tree again splits the data based on conditions
- Final leaf node gives a **numeric value**
- It usually chooses splits by reducing **variance** or **MSE**

Example:
Predict house price

- Area > 1500 sq ft
- Location = city
- Bedrooms > 2

Final output: **$320,000**

Loss:

- In regression, loss means **prediction error**
- We measure it using **MAE, MSE, RMSE**
- The model tries to make this loss as small as possible

ID3

- Used for **classification**
- Uses **Entropy** and **Information Gain**
- Chooses split with highest information gain
- Best for **categorical data**
- No pruning

C4.5

- Improved version of **ID3**
- Used for **classification**
- Uses **Gain Ratio**
- Handles **continuous + categorical data**
- Handles missing values
- Uses pruning

CART

- Used for **classification and regression**
- Uses **Gini** for classification
- Uses **MSE/variance reduction** for regression
- Always gives **binary split** (2 branches)

Cross-validation

- Used to check how well a model performs
- Helps make sure the model is not just memorizing training data
- Gives a better estimate of model performance

Hold-out method

- Dataset is split into **training set** and **testing set**
- Common split: **80% train, 20% test**
- Model learns on training data
- Model is checked on testing data
- Simple and fast
- Problem: result may depend too much on one random split

K-fold cross-validation

- Dataset is divided into **K equal parts**
- Model trains on **K-1 folds** and tests on **1 fold**
- This repeats **K times**
- Each fold gets a chance to be the test set
- Final performance = **average of all K results**
- More reliable than hold-out method

Example

- If **K = 5**, data is split into 5 parts
- Train on 4 parts, test on 1 part
- Repeat 5 times
- Take average accuracy/error

Why we use them

- To evaluate model properly
- To reduce overfitting risk
- To choose the best model more confidently

Main difference

- **Hold-out** = one split only
- **K-fold** = multiple splits, more reliable

-

Hyperparameter – parameters we use in the Decision Tree

Eg:

```
# we are tuning three hyperparameters rigl
grid_param = {
    'criterion': ['gini', 'entropy'],
    'max_depth' : range(2,32,1),
    'min_samples_leaf' : range(1,10,1),
    'min_samples_split': range(2,10,1),
    'splitter' : ['best', 'random']

}
```

```
best_parameters = grid_search.best_params_
print(best_parameters)
{'criterion': 'entropy', 'max_depth': 24, 'min_samples_leaf': 1, 'min_samples_split': 2, 'splitter': 'random'}
```

After Grid search We find the best parameters

- Training = model learns from old data
- Inferencing = model uses that learning to give output for unseen data
- Fine-tuning = improving an already trained model further for a specific task
- Transfer learning - using knowledge learned from one task for another related task._ Try for my project

Ensemble learning means **combining multiple models to get better prediction**.

- Instead of using one model, we use **many models together**
- Final output is taken by **combining their results**
- It usually gives **better accuracy and stability**

Why we use it

- To improve performance
- To reduce overfitting
- To make predictions more robust
- To reduce weakness of a single model

Common types

- **Bagging** = models work in parallel, example: **Random Forest – Dr Strange time stone**
- **Boosting** = models learn one after another, example: **AdaBoost, XGBoost**
- **Voting** = multiple models vote for final class

Example

Suppose 3 models predict:

- Model 1 → Yes
- Model 2 → Yes
- Model 3 → No

Final prediction by voting = **Yes**

Simple idea

- One student may make mistakes
- Many students answering together gives a better answer

One-line note:
Ensemble learning = combining multiple ML models to improve prediction accuracy.

Random Forest is an **ensemble learning algorithm** built using **many decision trees**.

Small notes

- **Random Forest** uses multiple **decision trees**
- It is mainly based on **bagging**
- Each tree is trained on a **random sample of data**
- It also uses a **random subset of features**
- Final output is combined from all trees

How it works

- For **classification** → final answer is by **majority voting**
- For **regression** → final answer is by **average prediction**

Why we use it

- Reduces **overfitting** compared to one decision tree
- Gives **better accuracy**
- More **stable and robust**
- Handles large data well

Small example